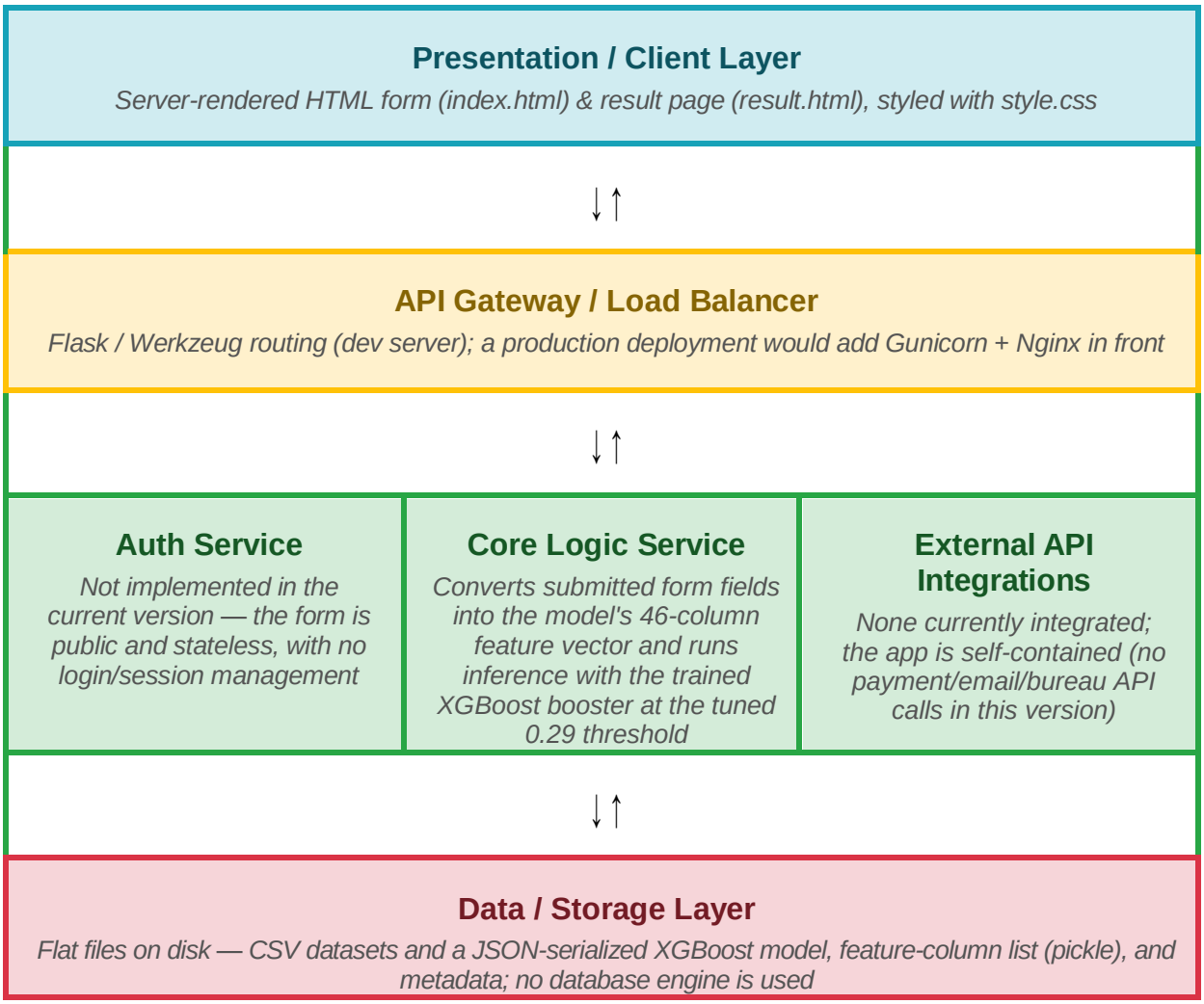
Solution Architecture

Date	07 July 2026
Team ID	xxxxxx
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Solution Architecture Diagram:

The project is implemented as a small, self-contained Flask application rather than a distributed microservice system. The diagram below maps that implementation onto the standard layered structure, and notes where a layer is not part of the current scope.



Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Collects applicant details (age, gender, income, employment, housing, family status, etc.) via a Jinja2-rendered HTML form, and displays the Approve/Reject decision with a probability gauge on the result page.	HTML5, Jinja2 templates, CSS (style.css)
API Gateway	Routes incoming GET/POST requests to the correct view function (index and /predict). In the current version this is Flask's own router; a production deployment would place a WSGI server and reverse proxy in front of it.	Flask routing, Werkzeug (dev server); Gunicorn + Nginx (recommended for production)
Microservices	Single-process service that builds the applicant's feature vector to match the model's training-time column order, loads the persisted XGBoost booster once at startup, and applies the tuned decision threshold to output a class and probability.	Python, Flask, XGBoost, pandas, joblib
Database	No relational or NoSQL database is used. Training data (application_record.csv, credit_record.csv, the cleaned/vintage-filtered dataset) and model artifacts (model JSON, feature-column pickle, metadata JSON) are stored as flat files on the local filesystem.	Local filesystem, CSV, JSON, pickle (joblib)